SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	**CSA12**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO5**	**Assessment:**	**ASSESSMENT TOOL2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)**
Weightage:	**30%**	**Total Marks:**	**25**
CO5 Statement:	*Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.*		
Student Name:	N.AKHIL KUMAR	**Reg.No.:**	192425128
Department:	AIML	**Date:**	

ANNEXURE A

Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.
2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.
3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.
4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.
5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Code of Conduct:

I N.AKHIL KUMAR(192425128) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *N.AKHIL KUMAR*

MARKS ALLOCATION

	Problem Understanding (20%)	Algorithm Design (30%)	Use of Control Structures (20%)	Pseudocode Structure (10%)	Completeness (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

ANSWERS:

Q1. DMA-Based Data Transfer Algorithm

Problem Understanding

Design a DMA-based algorithm that transfers data directly from an I/O device to main memory. The CPU should configure the DMA controller and handle completion/error interrupts, but should **not copy the actual data**.

Inputs

- I/O device
- Destination memory address
- Transfer size
- DMA controller
- Interrupt mechanism

Output

- Data successfully transferred to memory
- Transfer completion/error notification

Pseudocode

```
ALGORITHM DMA_Data_Transfer

INPUT:
   Device_ID
   Memory_Address
   Transfer_Size

OUTPUT:
   Transfer_Status

BEGIN

   INITIALIZE DMA_Controller
   INITIALIZE Interrupt_Controller

   // Step 1: Validate request
   IF Transfer_Size <= 0 THEN
      RETURN "Invalid transfer size"
   END IF

   IF Memory_Address is not valid THEN
      RETURN "Invalid memory address"
   END IF

   // Step 2: Prepare destination buffer
```

```
    Allocate Memory_Buffer of Transfer_Size

    // Step 3: Configure DMA
    DMA.Source      ← Device_ID
    DMA.Destination ← Memory_Buffer
    DMA.Length      ← Transfer_Size
    DMA.Direction   ← DEVICE_TO_MEMORY
    DMA.Mode        ← INTERRUPT_ON_COMPLETION

    // Step 4: Enable DMA interrupt
    Enable_Interrupt(DMA_INTERRUPT)

    // Step 5: Start DMA transfer
    DMA.Start()

    // CPU does NOT copy the data
    CPU_State ← AVAILABLE

    // Step 6: Wait for interrupt
    WHILE DMA.Transfer_Complete = FALSE DO

        IF DMA.Error = TRUE THEN
            Disable_Interrupt(DMA_INTERRUPT)
            RETURN "DMA transfer failed"
        END IF

        Perform_Other_CPU_Tasks()

    END WHILE

END
```

DMA Interrupt Service Routine

```
ISR DMA_Interrupt_Handler

BEGIN

    Save_Minimum_CPU_Context()

    Status ← DMA.Read_Status()

    IF Status = TRANSFER_COMPLETE THEN

        Clear_DMA_Interrupt()
```

```
        Mark_Buffer_Ready()

        Notify_Application()

    ELSE IF Status = TRANSFER_ERROR THEN

        Clear_DMA_Interrupt()

        Record_Error()

        Notify_Error_Handler()

    END IF

    Restore_CPU_Context()

    RETURN_FROM_INTERRUPT()

END ISR
```

Optimized Operation

I/O Device
↓
DMA Controller
↓
Main Memory
↓
DMA Completion Interrupt
↓
CPU

The CPU only performs **configuration and completion processing**, while DMA handles the bulk transfer.

Why this achieves Level 4

- Uses DMA correctly.
- CPU involvement is minimized.
- Includes interrupt handling.
- Handles errors.
- Uses conditions and loops.
- Separates normal processing from the ISR.
- Avoids CPU-based byte-by-byte copying.

Q2. Priority-Based Vectored Interrupt Handling

Problem Understanding

Multiple devices generate interrupts simultaneously. The system must determine which interrupt has the highest priority, identify the correct service routine using a vector table, execute it, and then return to the interrupted task.

Inputs

- Interrupt requests from multiple devices.
- Priority levels.
- Interrupt vector table.

Output

- Correct ISR executed for each interrupt.

Pseudocode

```
ALGORITHM Vectored_Interrupt_Controller

INPUT:
   Interrupt_Request_List

OUTPUT:
   Completed_ISR

BEGIN

   Initialize_Interrupt_Controller()

   FOR EACH Device IN Device_List DO

      Assign_Vector(Device)
      Assign_Priority(Device)

   END FOR

   WHILE System_Running = TRUE DO

      Pending_List ← Get_Pending_Interrupts()

      IF Pending_List is NOT EMPTY THEN

         // Select highest-priority interrupt
         Highest_Priority ← Find_Highest_Priority(Pending_List)

         Vector_Number ← Get_Vector_Number(Highest_Priority)

         Dispatch_ISR(Vector_Number)
```

```
        ELSE

            Execute_Normal_Task()

        END IF

    END WHILE

END
```

Interrupt Dispatcher

```
FUNCTION Dispatch_ISR(Vector_Number)

BEGIN

    Save_CPU_Context()

    ISR_Address ← Interrupt_Vector_Table[Vector_Number]

    IF ISR_Address is NULL THEN

        Record_Invalid_Interrupt(Vector_Number)

        Restore_CPU_Context()

        RETURN

    END IF

    Execute ISR_Address

    Restore_CPU_Context()

    RETURN

END FUNCTION
```

Example Priority Table

Device	Vector	Priority
Emergency Sensor	1	1
Network Controller	2	2
DMA Controller	3	3
USB Controller	4	4

Device	Vector	Priority
Keyboard	5	5

Here, **priority 1 is the highest priority**.
If several devices interrupt simultaneously:
Sensor + USB + DMA
↓
Priority Controller
↓
Sensor selected
↓
Vector 1
↓
Sensor_ISR()
Nested Interrupt Extension
ISR Current_Device

BEGIN

Save_Context()

Enable_Higher_Priority_Interrupts()

Handle_Device()

Disable_Higher_Priority_Interrupts()

Restore_Context()

RETURN

END
A higher-priority interrupt can therefore interrupt a lower-priority ISR.
Why this achieves Level 4
The algorithm includes:

- Interrupt prioritization.
- Vector-table lookup.
- ISR dispatch.
- Invalid interrupt handling.
- Nested interrupt capability.
- Loops and conditional statements.
- Normal task execution when no interrupt is pending.

Q3. Dynamic Selection Between Memory-Mapped I/O and I/O-Mapped I/O

Problem Understanding

Different devices may have different latency and bandwidth requirements. The system should select an appropriate I/O mechanism based on workload characteristics.

Inputs

- Required latency.
- Required bandwidth.
- Processor architecture support.
- Device type.
- Access frequency.

Output

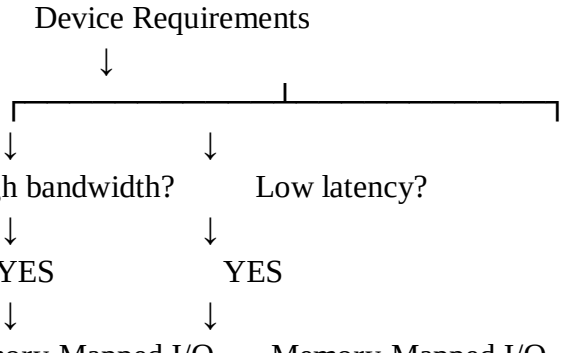
- Selected I/O mechanism.

Pseudocode

```
ALGORITHM Select_IO_Mechanism

INPUT:
   Required_Latency
   Required_Bandwidth
   Access_Frequency
   Device_Type
   CPU_Supports_IO_Mapped
   CPU_Supports_Memory_Mapped

OUTPUT:
   Selected_IO_Method

BEGIN

   // Check whether memory-mapped I/O is supported
   IF CPU_Supports_Memory_Mapped = FALSE THEN

      IF CPU_Supports_IO_Mapped = TRUE THEN
         RETURN "I/O-MAPPED I/O"
      ELSE
         RETURN "No supported I/O mechanism"
      END IF

   END IF

   // High-performance devices
   IF Required_Bandwidth >= HIGH_BANDWIDTH THEN
```

```
        Selected_IO_Method ← "MEMORY-MAPPED I/O"

    // Very latency-sensitive access
    ELSE IF Required_Latency <= LOW_LATENCY THEN

        Selected_IO_Method ← "MEMORY-MAPPED I/O"

    // Frequently accessed device registers
    ELSE IF Access_Frequency >= HIGH_ACCESS_FREQUENCY THEN

        Selected_IO_Method ← "MEMORY-MAPPED I/O"

    // Legacy/separate I/O-space architecture
    ELSE IF CPU_Supports_IO_Mapped = TRUE AND
            Device_Type = "LEGACY_IO_DEVICE" THEN

        Selected_IO_Method ← "I/O-MAPPED I/O"

    ELSE

        Selected_IO_Method ← "MEMORY-MAPPED I/O"

    END IF

    RETURN Selected_IO_Method

END
```

Decision Logic



For a device that uses a dedicated I/O address space on a processor that supports it:

Legacy Device

↓

Separate I/O Address Space

↓

I/O-Mapped I/O

Important Design Note

The selection should not be based on an assumption that one mechanism is universally faster. Actual latency depends on the **processor architecture, bus/interconnect, compiler instructions, device implementation, and system configuration**.

Therefore, a production implementation should measure actual performance.

Benchmark Extension

```
FUNCTION Measure_IO_Performance(Method)

BEGIN

   Start_Timer()

   FOR i ← 1 TO TEST_ITERATIONS DO
      Perform_IO_Access(Method)
   END FOR

   Stop_Timer()

   Latency ← Calculate_Average_Latency()

   Bandwidth ← Calculate_Bandwidth()

   RETURN Latency, Bandwidth

END FUNCTION
```

The system can compare measured results before selecting the configuration.

Q4. Bus Arbitration for PCIe, USB, and Thunderbolt

Problem Understanding

Multiple devices may request access to shared system resources. The arbitration mechanism should prevent one device from monopolizing available resources while ensuring that high-bandwidth devices receive sufficient service.

The system contains:

- PCIe devices.
- USB devices.
- Thunderbolt devices.

The algorithm must provide:

- Fairness.
- Priority handling.
- Starvation prevention.
- Queue management.
- Resource allocation.

Pseudocode

```
ALGORITHM Bus_Arbitration

INPUT:
   PCIe_Queue
   USB_Queue
   Thunderbolt_Queue

OUTPUT:
   Device_Service

BEGIN

   Initialize_Queues()

   Set_Weight(PCIe, 3)
   Set_Weight(USB, 1)
   Set_Weight(Thunderbolt, 3)

   WHILE System_Running = TRUE DO

      Update_Request_Queues()

      // First handle urgent requests
      IF Exists_High_Priority_Request() THEN

         Device ← Select_High_Priority_Request()

         Grant_Bus(Device)

         Service(Device)

         Release_Bus(Device)

      ELSE

         // Weighted fair arbitration
         Device ← Weighted_Round_Robin(
                  PCIe_Queue,
                  USB_Queue,
                  Thunderbolt_Queue)

         IF Device != NONE THEN
```

```
            Grant_Bus(Device)

            Service(Device)

            Release_Bus(Device)

        ELSE

            Bus_State ← IDLE

        END IF

    END IF

    Update_Starvation_Counters()

    Prevent_Starvation()

  END WHILE

END
```

Weighted Round-Robin

Example:

PCIe → 3 service opportunities

USB → 1 service opportunity

Thunderbolt → 3 service opportunities

The weights should be configurable according to system requirements.

Starvation Prevention

```
FUNCTION Prevent_Starvation()

BEGIN

  FOR EACH Device DO

    IF Device.Wait_Time > MAX_WAIT_TIME THEN

      Device.Priority ← Device.Priority + BOOST

      Add_To_Urgent_Queue(Device)

    END IF
```

END FOR

END FUNCTION

This ensures that a lower-bandwidth USB device cannot wait indefinitely while high-speed devices continuously request service.

Bus Allocation

Request
↓
Queue
↓
Priority Check
↓
Starvation Check
↓
Weighted Arbitration
↓
Grant Bus
↓
Transfer
↓
Release Bus

Why this achieves Level 4

The algorithm contains:

- Multiple device queues.
- Priority handling.
- Weighted arbitration.
- Starvation prevention.
- Loops.
- Conditional statements.
- Configurable policies.

It balances **performance and fairness** rather than giving unlimited priority to high-speed devices.

Q5. Hybrid I/O Communication Mechanism

Problem Understanding

Different devices have different data rates.

Low-speed devices do not justify the overhead of DMA setup, while high-speed devices generate enough data to make CPU-driven polling inefficient.

Therefore, the system uses:

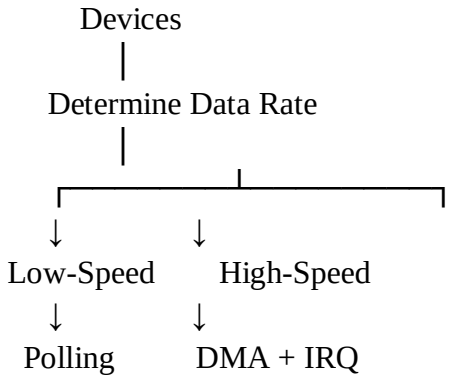
Low-speed devices

Polling

High-speed devices

DMA + interrupts

2. Device Classification



3. Pseudocode

```
ALGORITHM Hybrid_IO_Controller

INPUT:
   Device_List

OUTPUT:
   Device_Data

BEGIN

   Initialize_IO_System()

   FOR EACH Device IN Device_List DO

      IF Device.Data_Rate <= LOW_SPEED_THRESHOLD THEN

         Device.Mode ← POLLING

      ELSE

         Device.Mode ← DMA_INTERRUPT

         Configure_DMA(Device)

         Enable_DMA_Interrupt(Device)

      END IF

   END FOR
```

```
    WHILE System_Running = TRUE DO

        // Handle low-speed devices
        FOR EACH Device IN Device_List DO

            IF Device.Mode = POLLING THEN

                IF Device.Is_Ready() = TRUE THEN

                    Data ← Device.Read()

                    Process_Data(Device, Data)

                END IF

            END IF

        END FOR

        Perform_Normal_CPU_Tasks()

    END WHILE

END
```

4. DMA Interrupt Handler

```
ISR DMA_Completion_Handler(Device_ID)

BEGIN

    Save_Minimum_Context()

    Status ← DMA.Get_Status(Device_ID)

    IF Status = TRANSFER_COMPLETE THEN

        Buffer ← DMA.Get_Completed_Buffer(Device_ID)

        Mark_Buffer_Ready(Device_ID, Buffer)

        Queue_For_Processing(Device_ID, Buffer)
```

```
        Clear_DMA_Interrupt(Device_ID)

    ELSE IF Status = TRANSFER_ERROR THEN

        Record_DMA_Error(Device_ID)

        Restart_DMA(Device_ID)

    END IF

    Restore_Minimum_Context()

    RETURN_FROM_INTERRUPT()

END ISR
```

5. Polling Mechanism

```
FUNCTION Poll_Low_Speed_Device(Device)

BEGIN

    IF Device.Status_Register = READY THEN

        Data ← Read_Device_Register(Device)

        Process_Data(Device, Data)

        Clear_Device_Status(Device)

    ELSE

        RETURN

    END IF

END FUNCTION
```

Polling is appropriate when:

- Data arrives infrequently.
- Device status checks are inexpensive.
- Interrupt setup overhead would not provide significant benefit.

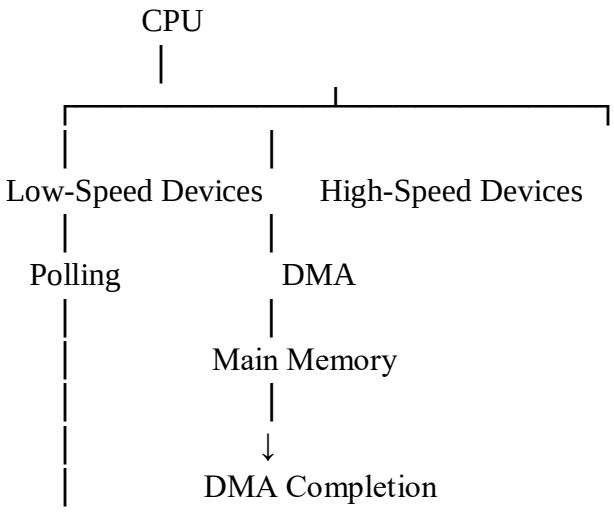
6. Dynamic Mode Switching

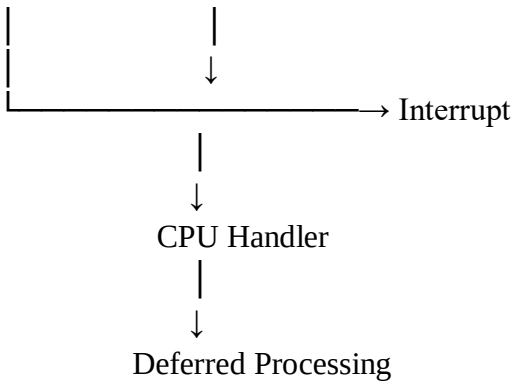
A stronger implementation can dynamically change the device's I/O method based on measured traffic.

```
ALGORITHM Dynamic_Hybrid_Mode

INPUT:
   Device

BEGIN

   Data_Rate ← Measure_Data_Rate(Device)

   CPU_Load ← Measure_CPU_Load(Device)

   IF Data_Rate < LOW_SPEED_THRESHOLD AND
     CPU_Load < CPU_LIMIT THEN

      Device.Mode ← POLLING

   ELSE

      Device.Mode ← DMA_INTERRUPT

      Configure_DMA(Device)

      Enable_Interrupt(Device)

   END IF

END
```

If a device suddenly begins generating large amounts of data, the system can switch it from polling to DMA.

7. Overall Hybrid Architecture




Interrupt
CPU Handler
Deferred Processing